

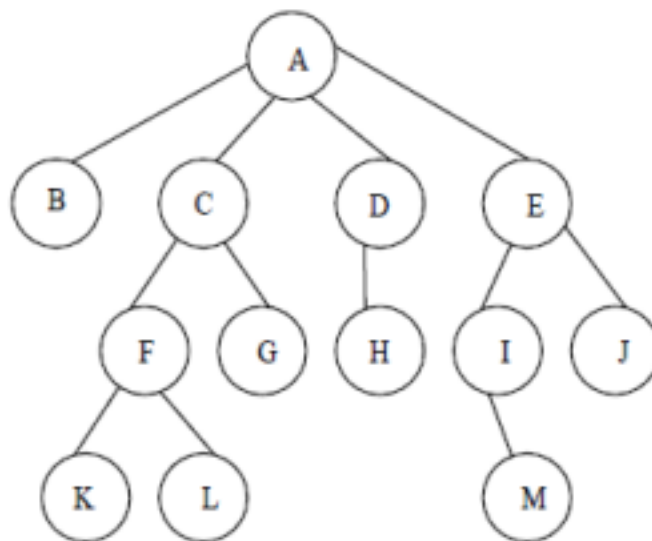
CS3301 DATA STRUCTURES

UNIT III TREES CS3301 Data Structures Important Questions

Tree ADT – Tree Traversals – Binary Tree ADT – Expression trees – Binary Search Tree ADT – AVL Trees – Priority Queue (Heaps) – Binary Heap.

3.1 TREE ADT

TREE : A tree is a finite set of one or more nodes such that there is a specially designated node called the Root, and zero or more non empty sub trees T₁, T₂T_k, each of whose roots are connected by a directed edge from Root R.



NODE : Item of Information.

ROOT : A node which does not have a parent. Here, Root is A. **LEAF :** A node which does not have children is called leaf or Terminal node. Here B, K, L, G, H, M, J are leafs.

SIBLINGS : Children of the same parents are said to be siblings, Here B, C, D, E are siblings, F, G are siblings. Similarly I, J & K, L are siblings. **PATH :** A path from node n₁ to n_k is defined as a sequence of nodes n₁, n₂, n₃ n_k such that n_i is the parent of n_{i+1}. for 1 = i to k . There is exactly only one path from each node to root.

Here, path from A to L is A, C, F, L. where A is the parent for C, C is the parent of F and F is the parent of L.

LENGTH : The length is defined as the number of edges on the path. Here, the length for the path A to L is 3.

DEGREE : The number of sub trees of a node is called its degree. Here Degree of A is 4 Degree of C is 2

The degree of the tree is the maximum degree of any node in the tree. Here, the degree of the tree is 4.

LEVEL : The level of a node is defined by initially letting the root be at level one, if a node is at level L then its children are at level $L + 1$. Level of A is 1.

Level of B, C, D, is 2. Level of F, G, H, I, J is 3 Level of K, L, M is 4. **DEPTH :**

For any node n, the depth of n is the length of the unique path from root to n. The depth of the root is zero. Here, Depth of node F is 2. Depth of node L is 3.

HEIGHT : For any node n, the height of the node n is the length of the longest path from n to the leaf. The height of the leaf is zero Here, Height of node F is 1. Height of L is 0.

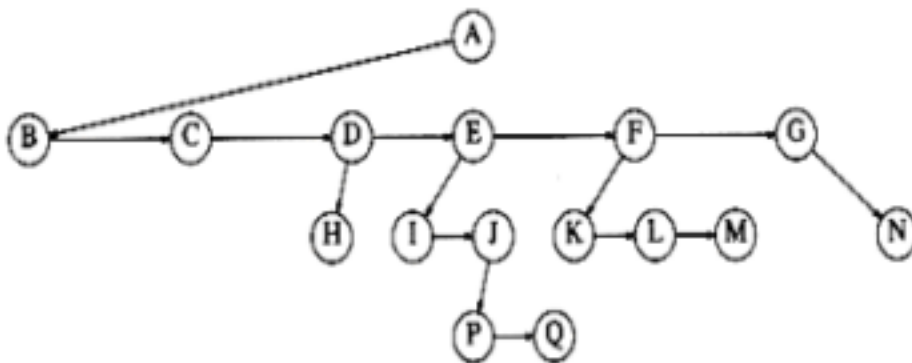
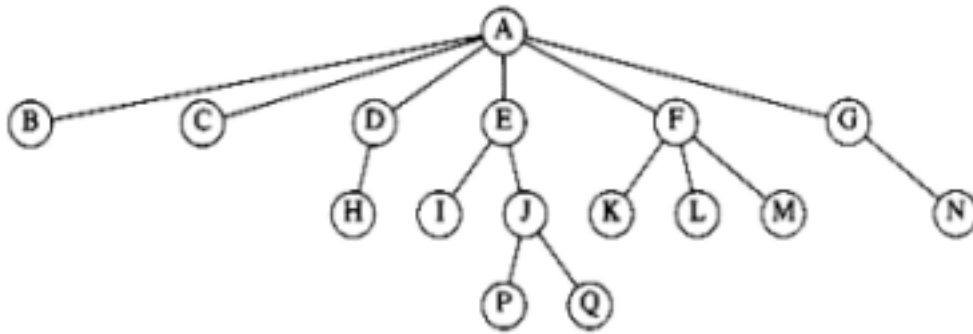
- The height of the tree is equal to the height of the root
- Depth of the tree is equal to the height of the tree.

Implementation of Trees

One way to implement a tree is create a node and in each node, store data, a pointer to each child of the node. Keep the children of each node in a linked list of tree nodes.

```
struct tree_node
{
    element_type element;
    tree_ptr first_child;
    tree_ptr next_sibling;
};
```

First child/next sibling representation of the tree



3.2. TREE TRAVERSALS

- Traversing means visiting each node only once.
- Tree traversal is a method for visiting all the nodes in the tree exactly once.

There are three types of tree traversal techniques

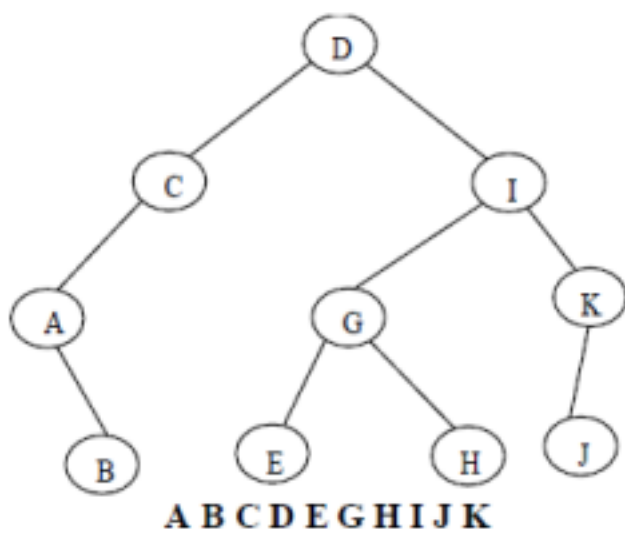
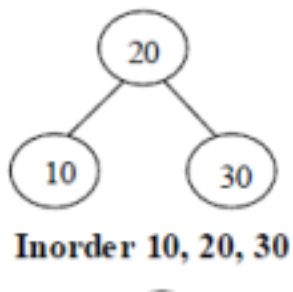
- a) Inorder Traversal
- b) Preorder Traversal
- c) Postorder Traversal

a) Inorder Traversal

The inorder traversal of a binary tree is performed as

- Traverse the left subtree in inorder
- Visit the root
- Traverse the right subtree in inorder.

Example :



Recursive routine for inorder traversal

```
void Inorder (Tree T)
{
if (T != NULL)
{
Inorder (T -> left);
printf("%d",T -> Element);
Inorder (T -> right);
}
}
```

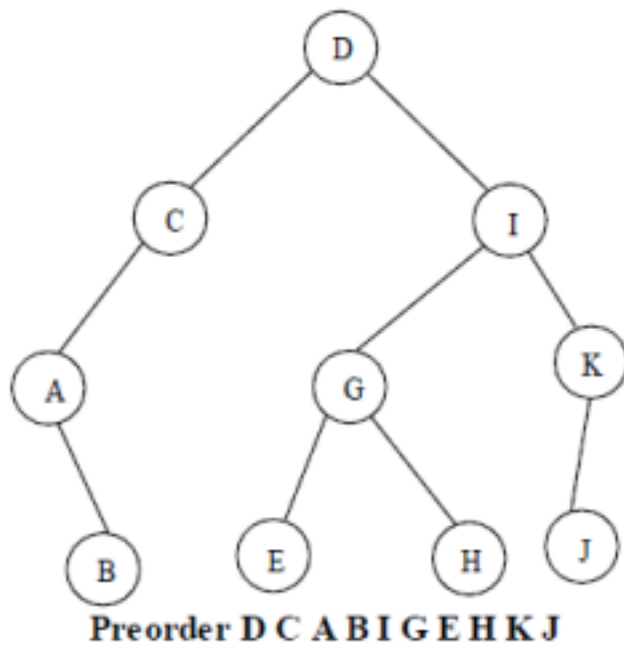
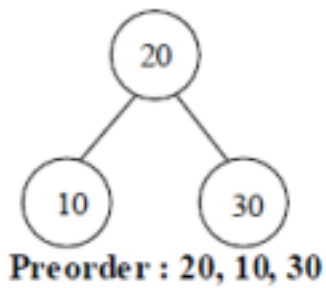
b) Preorder Traversal

The preorder traversal of a binary tree is performed as follows, • Visit the root

- Traverse the left subtree in preorder
- Traverse the right subtree in preorder.

Meenakshi Page 4

Example:



Recursive Routine For Preorder Traversal

```
void Preorder (Tree T)
{
if (T != NULL)
{
printf("%d", T -> Element);
Preorder (T -> left);
```

Preorder (T -> right);

}

}

c)Postorder Traversal

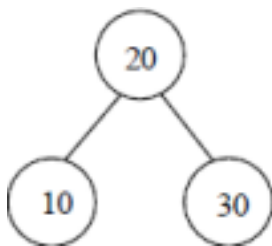
The postorder traversal of a binary tree is performed by the following steps. • Traverse the left subtree in postorder.

Meenakshi Page 5

• Traverse the right subtree in postorder. •

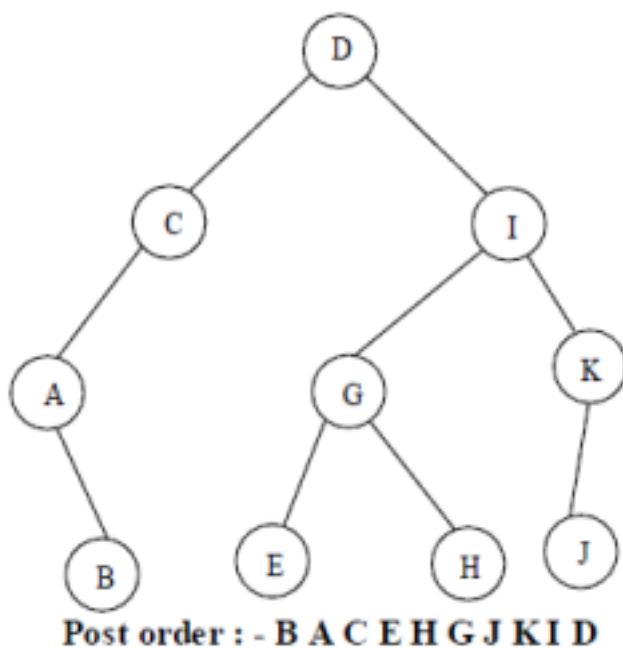
Visit the root.

Example : 1



Postorder : - 10, 30, 20

Example : 2



Recursive Routine For Postorder Traversal void Postorder (Tree T)

{

if (T != NULL)

{

```

Postorder (T -> Left);
Postorder (T -> Right); printf("%d",T ->
Element); }
}

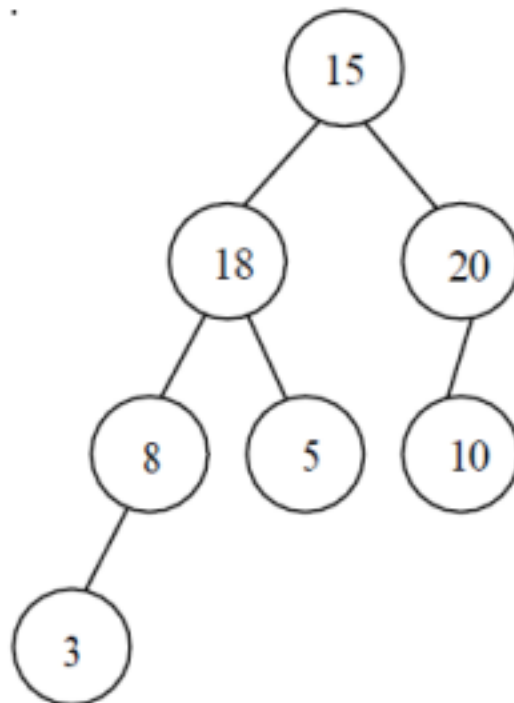
```

Meenakshi Page 6

3.3 Binary Tree ADT

- Binary Tree is a tree in which no node can have more than two children. •

Maximum number of nodes in the tree with level i is $2^i - 1$. (Assume root node is at level 1)



Binary Tree Node Declarations

```

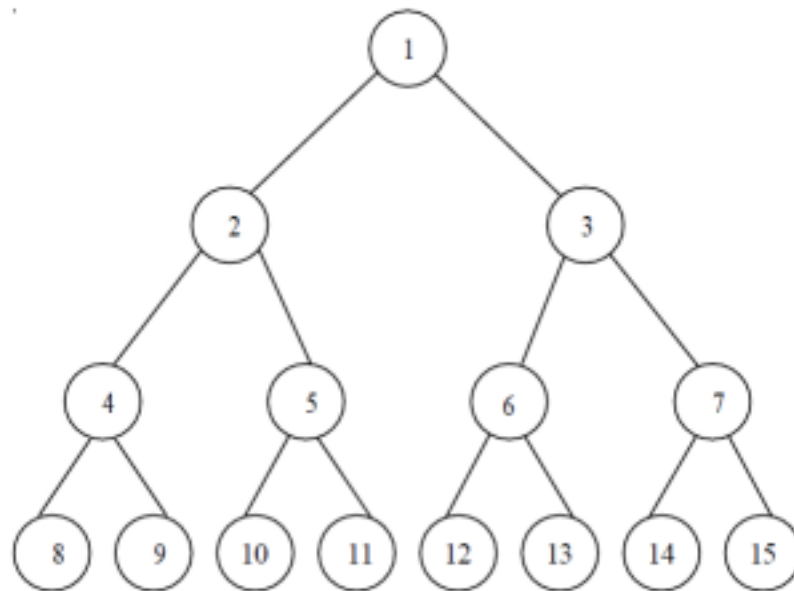
struct TreeNode
{
    int Element;
    struct TreeNode *Left ;
    struct TreeNode *Right;
};

```

Full Binary Tree

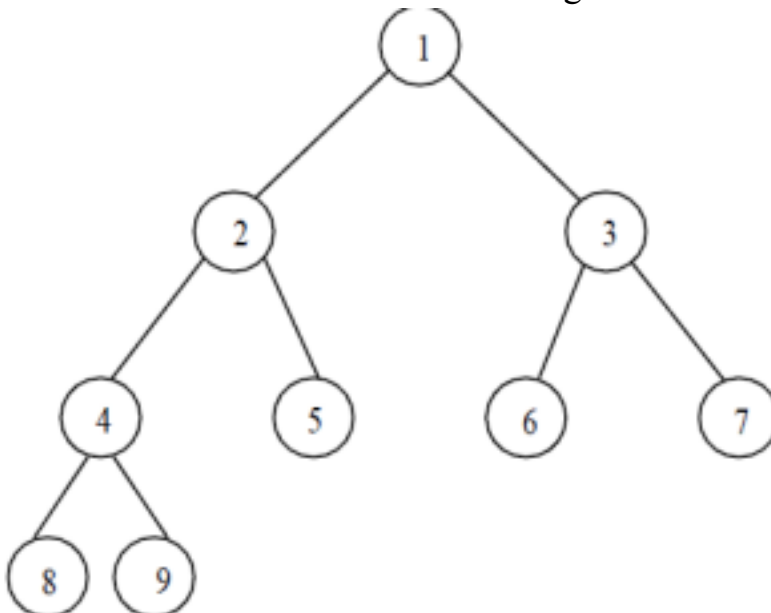
- A full binary tree of height h has $2^{h+1} - 1$ nodes.
- Here height is 3.
- No. of nodes in full binary tree is $= 2^{3+1} - 1 = 15$ nodes.
- A full binary tree (sometimes proper binary tree or 2-tree) is a tree in which every node other than the leaves has two children.

Meenakshi Page 7



Complete Binary Tree:

- A binary tree is said to be a complete binary tree when all the levels are completely filled except the last level, which is filled from the left
- A complete binary tree of height h has between 2^h and $2^{h+1} - 1$ nodes.
- In the bottom level the elements should be filled from left to right.



- A full binary

tree can be a complete binary tree, but all complete binary tree is not a full binary tree.

Representation of a Binary Tree

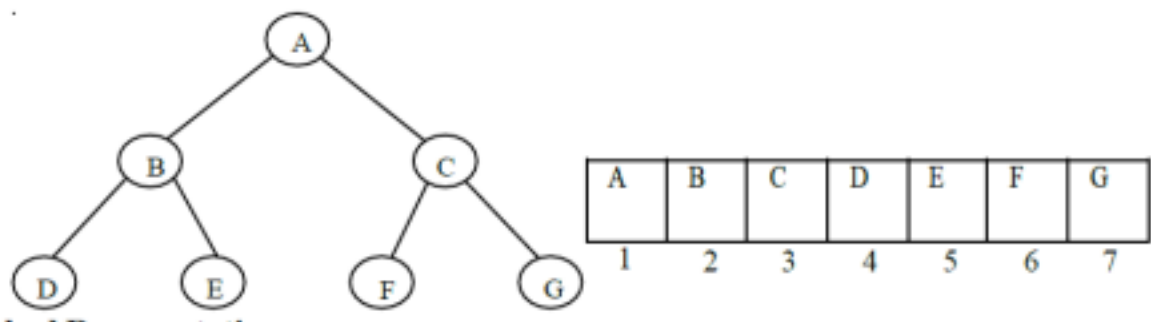
There are two ways for representing binary tree, they are

- Linear Representation
- Linked Representation

Meenakshi Page 8

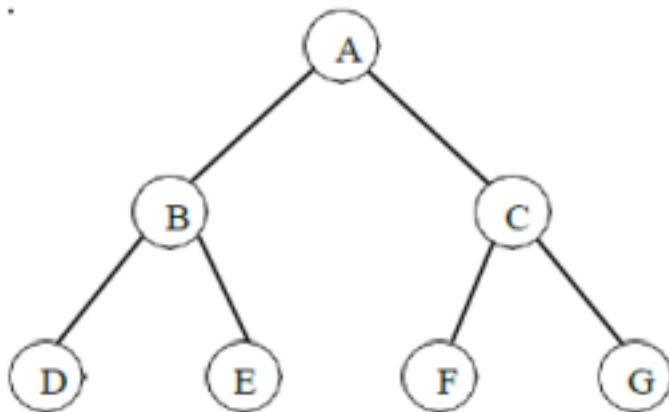
Linear Representation

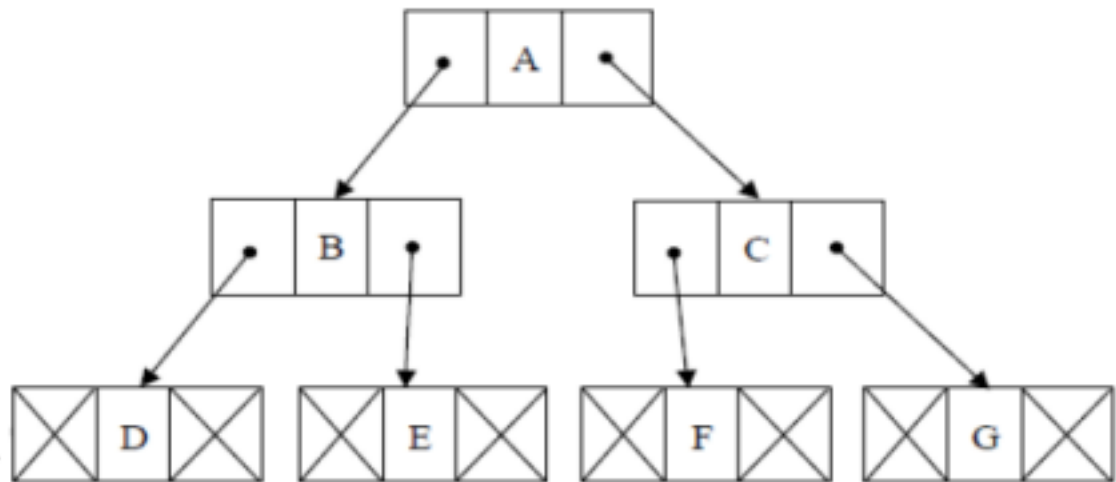
- The elements are represented using arrays.
- For any element in position i , the left child is in position $2i$, the right child is in position $(2i + 1)$, and the parent is in position $(i/2)$.



Linked Representation

- The elements are represented using pointers.
- Each node in linked representation has three fields, namely,
 - * Pointer to the left subtree
 - * Data field
 - * Pointer to the right subtree
- In leaf nodes, both the pointer fields are assigned as NULL.





The Leftmost-child, Right-sibling Data Structures

- In this representation, each node contains three fields namely, leftmost child, label and right sibling.
- Then, next pointers of node point to right siblings, and leftmost-child in cellspace.



3.4 Expression trees

- Expression Tree is a binary tree in which the leaf nodes are operands and the interior nodes are operators.
- Like binary tree, expression tree can also be traversed by inorder, preorder

and postorder traversal.

Meenakshi Page 10

Constructing an Expression Tree

Let us consider postfix expression given as an input for constructing an expression tree

1. Read one symbol at a time from the postfix expression.
2. Check whether the symbol is an operand or operator.

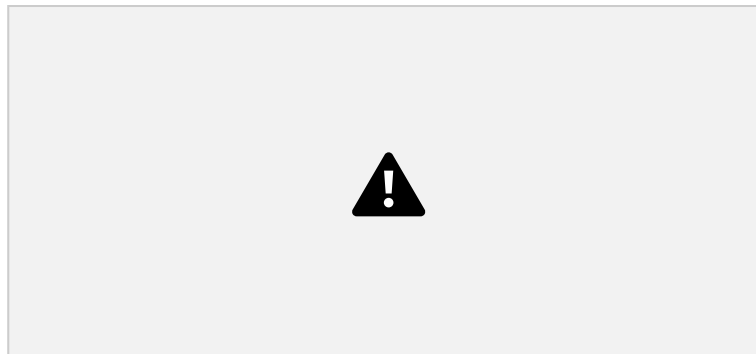
(a) If the symbol is an operand, create a one - node tree and push a pointer on to the stack.

(b) If the symbol is an operator pop two pointers from the stack namely T1 and T2 and form a new tree with root as the operator and T2 as a left child and T1 as a right child. A pointer to this new tree is then pushed onto the stack. **Example:**

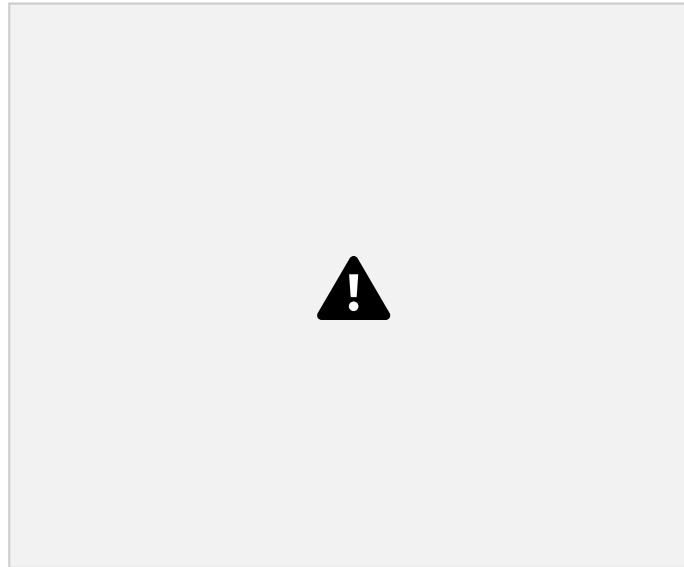
-

$ab + c *$

The first two symbols are operand, so create a one node tree and push the pointer on to the stack.

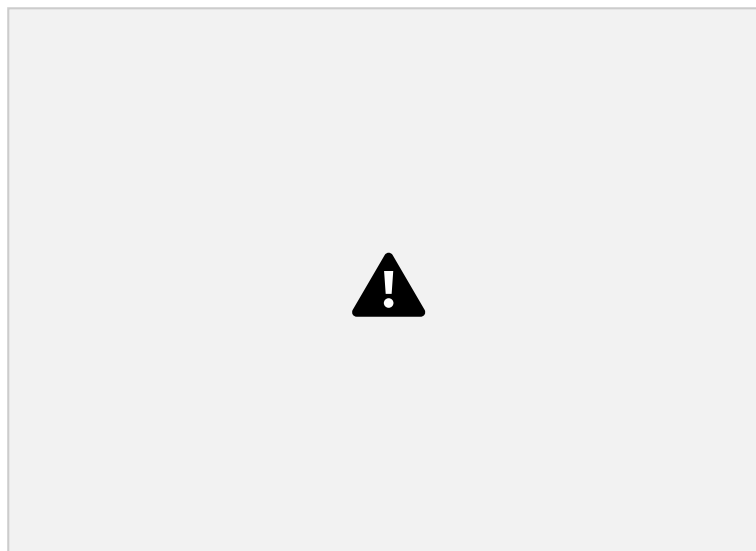


Next „+“ symbol is read, so two pointers are popped, a new tree is formed and a pointer to this is pushed on to the stack.



Meenakshi Page 11

Next the operand C is read, so a one node tree is created and the pointer to it is pushed onto the stack.



Now „*“ is read, so two trees are merged and the pointer to the final tree is pushed onto the stack



APPLICATIONS OF TREES

- Binary Search Tree - Used in many search applications where data is constantly entering/ leaving
- Binary Space Partition - Used in almost every 3D video game to determine what objects need to be rendered.
- Binary Tries - Used in almost every high-bandwidth router for storing router-tables.

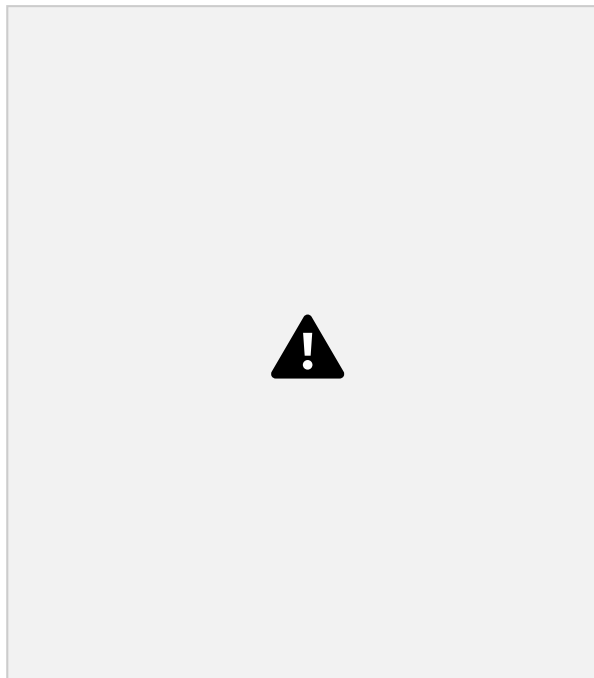
Meenakshi Page 12

- Hash Trees - used in p2p programs and specialized image-signatures in which a hash needs to be verified, but the whole file is not available.
- Heaps - Used in implementing efficient priority-queues,
- Huffman Coding Tree - used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats
- GGM Trees - Used in cryptographic applications to generate a tree of pseudo-random numbers.
- Syntax Tree - Constructed by compilers and (implicitly) calculators to parse expressions.
- Treap - Randomized data structure used in wireless networking and memory allocation.

- T-tree - Though most databases use some form of B-tree to store data on the drive, databases which keep all (most) their data in memory often use T trees to do so.
- BTree :We use BTree in indexing large records in database to improve search.

3. 5 Binary Search Tree ADT

In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.



Meenakshi Page 13

- Every BST is a Binary Tree
- But Not every binary tree is a BST

Declaration Routine for Binary Search Tree

```
struct TreeNode
```

```
{
    int Element ;
    struct TreeNode * Left;
    struct TreeNode * Right;
};
```

```
typedef struct TreeNode * SearchTree;
```

Make Empty :-

This operation is mainly for initialization when the programmer prefers to initialize the first element as a one - node tree.

Routine to Make an Empty Tree :-

SearchTree MakeEmpty (SearchTree T)

```
{
    if (T != NULL)
    {
        MakeEmpty (T -> left);
        MakeEmpty (T -> Right);
        free (T);
    }
    return NULL ;
}
```

Insert : -

To insert the element X into the tree,

- * Check with the root node T

- * If it is less than the root,

 Traverse the left subtree recursively until it reaches the T -> left equals to NULL. Then X is placed in T -> left.

- * If X is greater than the root.

Meenakshi Page 14

 Traverse the right subtree recursively until it reaches the T -> right equals to NULL. Then X is placed in T-> Right.

Routine to Insert into a Binary Search Tree

SearchTree Insert (int X, searchTree T)

```
{
    if (T == NULL)
    {
        T = malloc (size of (Struct TreeNode));
    }
}
```

```

    if (T! = NULL) // First element is placed in the root.
    {
        T -> Element = X;
        T -> left = NULL;
        T -> Right = NULL;
    }
}

else

    if (X < T -> Element)
        T -> left = Insert (X, T -> left);

    else

        if (X > T -> Element)
            T -> Right = Insert (X, T -> Right);

// Else X is in the tree already.
return T;

}

```





AFTER 3

Find : -

- Check whether the root is NULL if so then return NULL. • Otherwise, Check the value X with the root node value (i.e. T -> data) (1) If X is equal to T -> data, return T.
- (2) If X is less than T -> data, Traverse the left of T recursively. (3) If X is greater than T -> data, traverse the right of T recursively.

Routine for find Operation

SearchTree Find (int X, SearchTree T)

```
{
    if (T == NULL)
        return NULL ;
    if (X < T -> Element)
        return Find (X, T -> left);
    else
        if (X > T -> Element)
            return Find (X, T -> Right);
        else
            return T; // returns the position of the search element.
}
```



Find Min :

- This operation returns the position of the smallest element in the tree.

- To perform FindMin, start at the root and go left as long as there is a left child. The stopping point is the smallest element.

Recursive routine to find minimum

SearchTree FindMin (SearchTree T)

```
{  
if (T == NULL)  
    return NULL ;  
else if (T -> left == NULL)  
    return T;  
else  
    return FindMin (T -> left);  
}
```



FindMax

FindMax routine return the position of largest elements in the tree. To perform a FindMax, start at the root and go right as long as there is a right child. The stopping point is the largest element.

Recursive routine to find maximum

SearchTree FindMax (SearchTree T)

```
{  
if (T == NULL)  
return NULL ;  
else if (T -> Right == NULL)  
return T;  
else  
return FindMax (T -> Right);  
}
```





Delete :

Deletion operation is the complex operation in the Binary search tree. To delete an element, consider the following three possibilities.

CASE 1: Node to be deleted is a leaf node (i.e) No children.

CASE 2: Node with one child.

CASE 3: Node with two children.

CASE 1 Node with no children (Leaf node)

If the node is a leaf node, it can be deleted immediately.



CASE 2 : - Node with one child

If the node has one child, it can be deleted by adjusting its parent pointer that points to its child node.



Case 3: Node with two children

It is difficult to delete a node which has two children. The general strategy is to replace the data of the node to be deleted with its smallest data of the right subtree and recursively delete that node.







Routine for deletion from BST

SearchTree Delete (int X, searchTree T)

{

int Tmpcell ;

if (T == NULL)

Error (“Element not found”);

```

else
if (X < T -> Element) // Traverse towards left
    T -> Left = Delete (X, T -> Left);
else
if (X > T -> Element) // Traverse towards right
    T -> Right = Delete (X, T -> Right);
// Found Element to be deleted
else
// Two children
if (T -> Left && T -> Right)
{
// Replace with smallest data in right subtree Tmpcell = FindMin (T ->
    Right); T -> Element = Tmpcell -> Element ;
    T -> Right = Delete (T -> Element, T -> Right);
}
else // one or zero children
{
    Tmpcell = T;
if (T -> Left == NULL)
    T = T -> Right;
else if (T -> Right == NULL)
    T = T -> Left ;
free (TmpCell);
}
return T;
}

```

3.6 AVL TREE

An AVL tree is a balanced binary search tree. In an AVL tree, balance factor of every node is either -1, 0 or +1. For every node in the tree, the heights of the two sub-trees may differ by at most one.

BALANCE FACTOR

The balance factor of a node is calculated by subtracting the height of its right sub-tree from the height of its left sub-tree.

Every node has a balance factor of -1 , 0 , or 1 .

BALANCE FACTOR = HEIGHT (LEFT SUB-TREE) – HEIGHT (RIGHT SUB-TREE)

ROTATIONS

Rotation is the process of moving the nodes to either left or right to make tree balanced , If the balance factor of any node in an AVL tree becomes less than -1 or greater than 1

There are four rotations and they are classified into two types.

AN AVL TREE BECOMES UNBALANCED WHEN ONE OF THE FOLLOWING CONDITIONS OCCUR

1. An insertion into the left subtree of left child of node n
2. An insertion into the right subtree of right child of node n
3. An insertion into the right subtree of left child of node n
4. An insertion into the left subtree of right child of node n

In Cases 1 and 2 insertions occur at outside (Left-Left or Right-Right) and the unbalance condition is balanced by Single Rotation

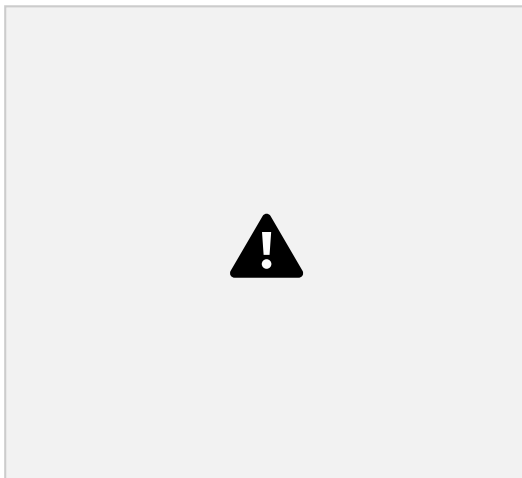
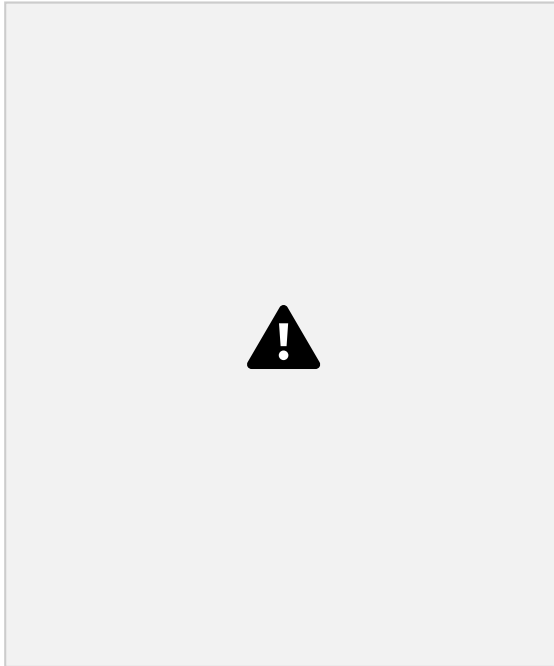
In Cases 3 and 4 insertions occur at inside (Left-Right or Right-Left) and the unbalance condition is balanced by Double Rotation

SINGLE ROTATION

Single rotation occurs for case 1&2 i.e

• An insertion into the left subtree of left child of node and (LL Rotation) • An insertion into the right subtree of right child of node n (RR Rotation) Case 1. An insertion into the left subtree of the left child of $K2$. Single Rotation to fix Case 1.

General Representation



Routine to Perform Single Rotation with Left

SingleRotatewithLeft (Position K2)

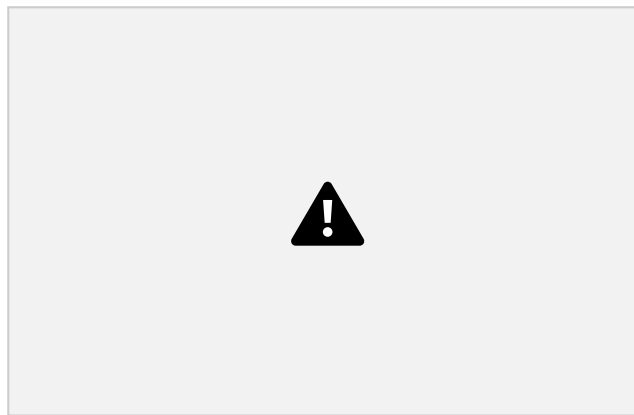
```
{  
Position K1;  
K1 = K2 -> Left ;  
K2 -> left = K1 -> Right ;  
K1 -> Right = K2 ;  
K2 -> Height = Max (Height (K2 -> Left), Height (K2 -> Right)) + 1 ;  
K1 -> Height = Max (Height (K1 -> left), Height (K1 -> Right)) + 1;  
return K1 ;
```

}

Meenakshi Page 27

Example :

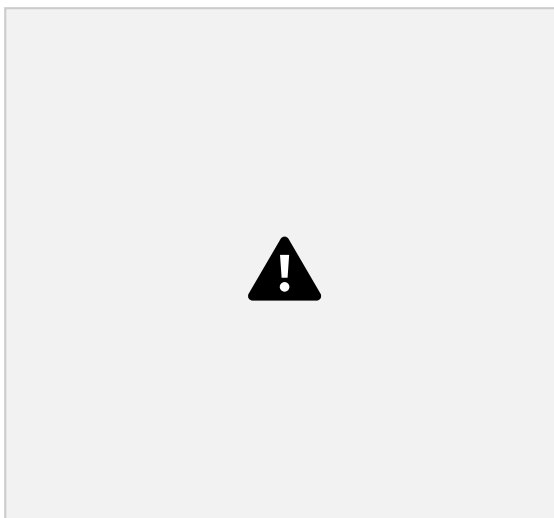
Inserting the value '1' in the following AVL Tree makes AVL Tree imbalance



Single Rotation to fix Case 4 :-

Case 4 : - An insertion into the right subtree of the right child of

K1 General Representation





Routine to Perform Single Rotation with Right :

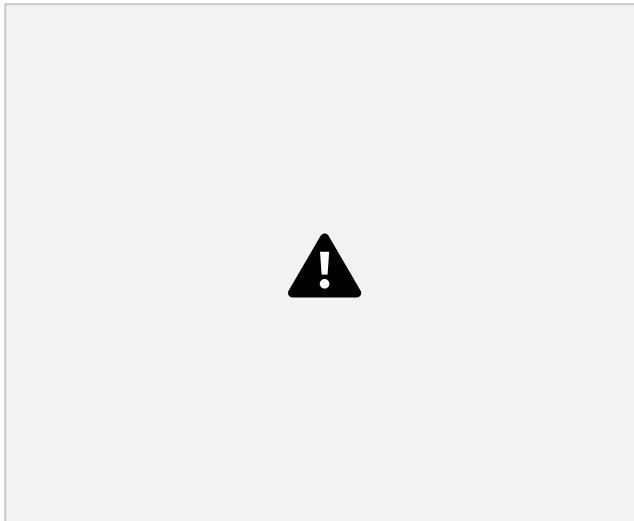
Single RotationWithRight (Position K1)

```
{  
Position K2 ;  
K2 = K1 -> Right;  
K1 -> Right = K2 -> Left ;  
K2 -> Left = K1 ;  
K2 -> Height = Max (Height (K2 -> Left), Height (K2 -> Right)) +1 ;  
K1 -> Height = Max (Height (K1 -> Left), Height (K1 -> Right)) +1 ;  
Return K2 ;  
}
```

Example : -

Inserting the value '10' in the following AVL Tree.

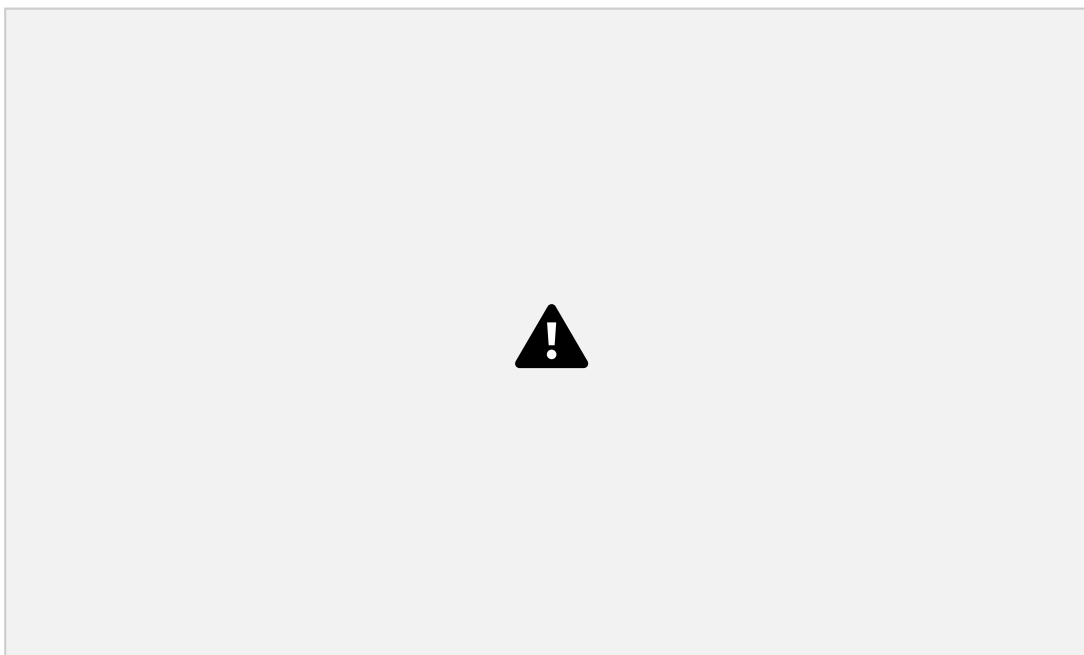




Double Rotation

Double Rotation is performed to fix case 2 and case 3.

Case 2 : An insertion into the right subtree of the left child. **General Representation**



This can be performed by 2 single rotations.

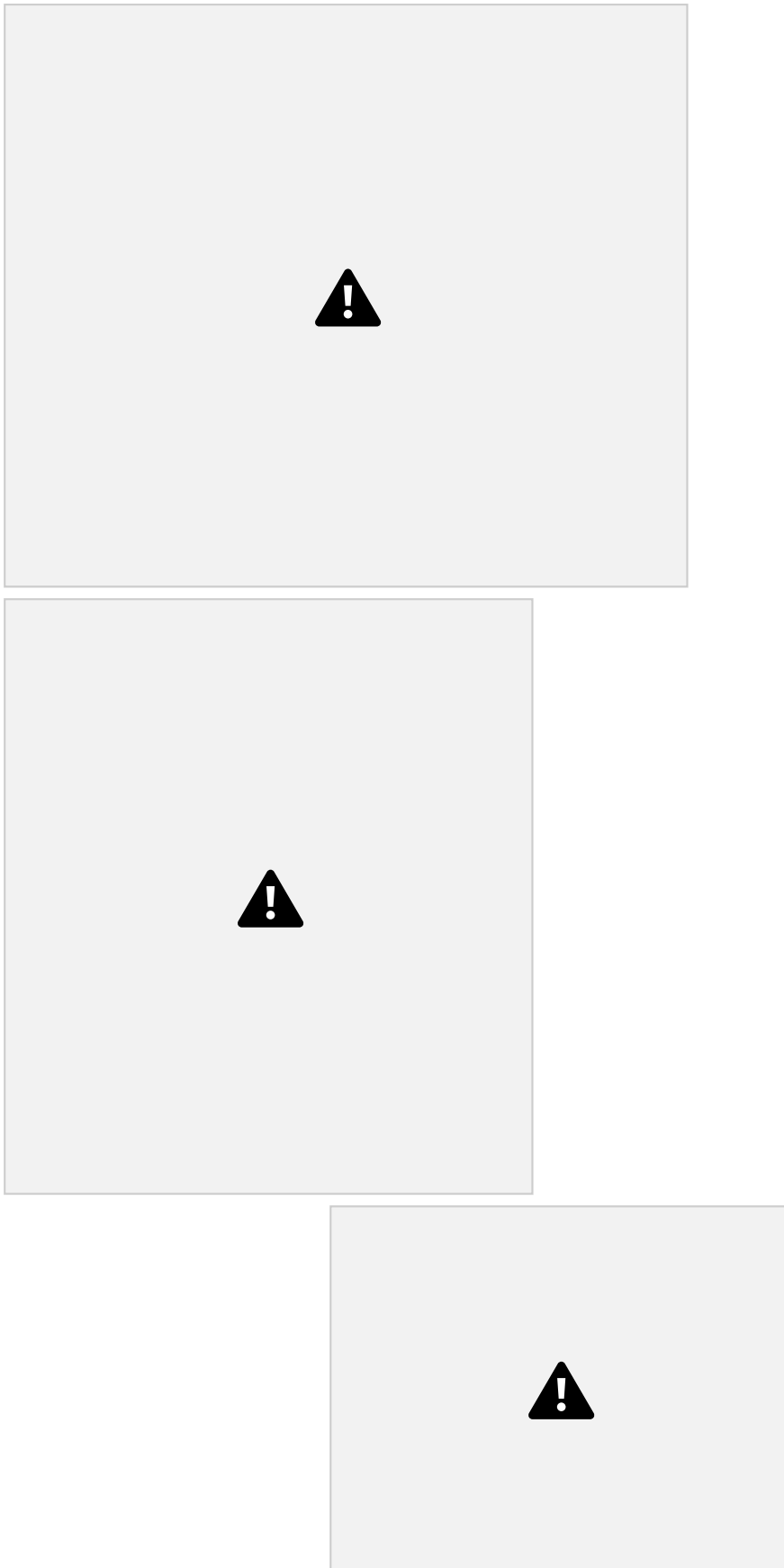


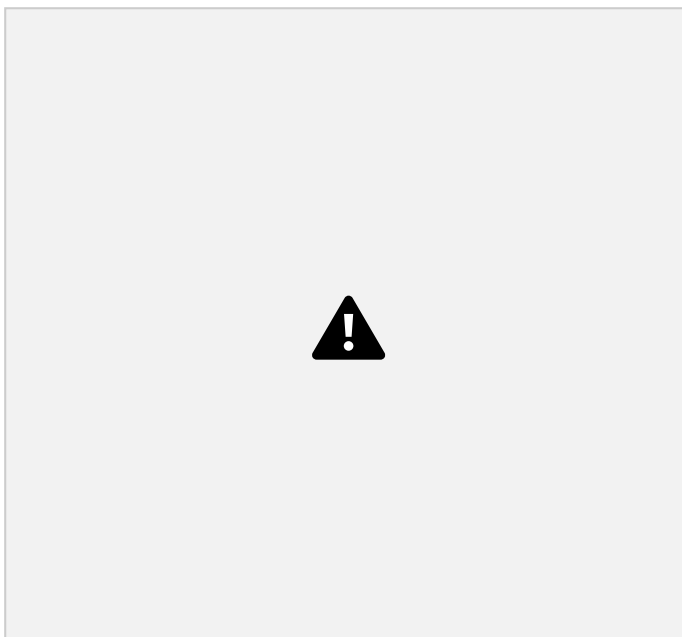
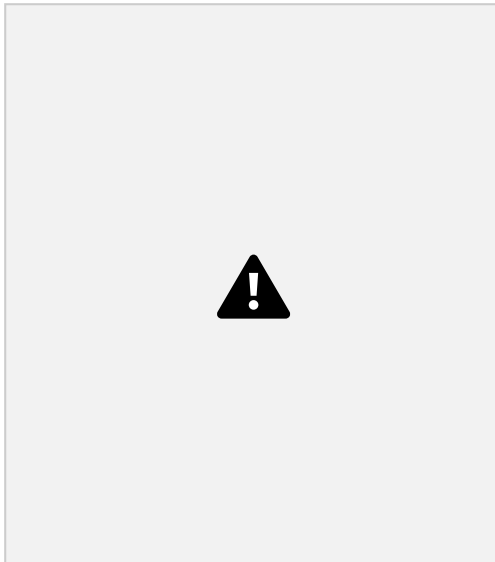
Fig. 3.8.7 Balanced AVL Tree

```

{ /* Rotation Between K1 & K2 */
    K3 -> Left = SingleRotateWithRight (K3 -> Left);
/* Rotation Between K3 & K2 */
    return SingleRotateWithLeft (K3);
}

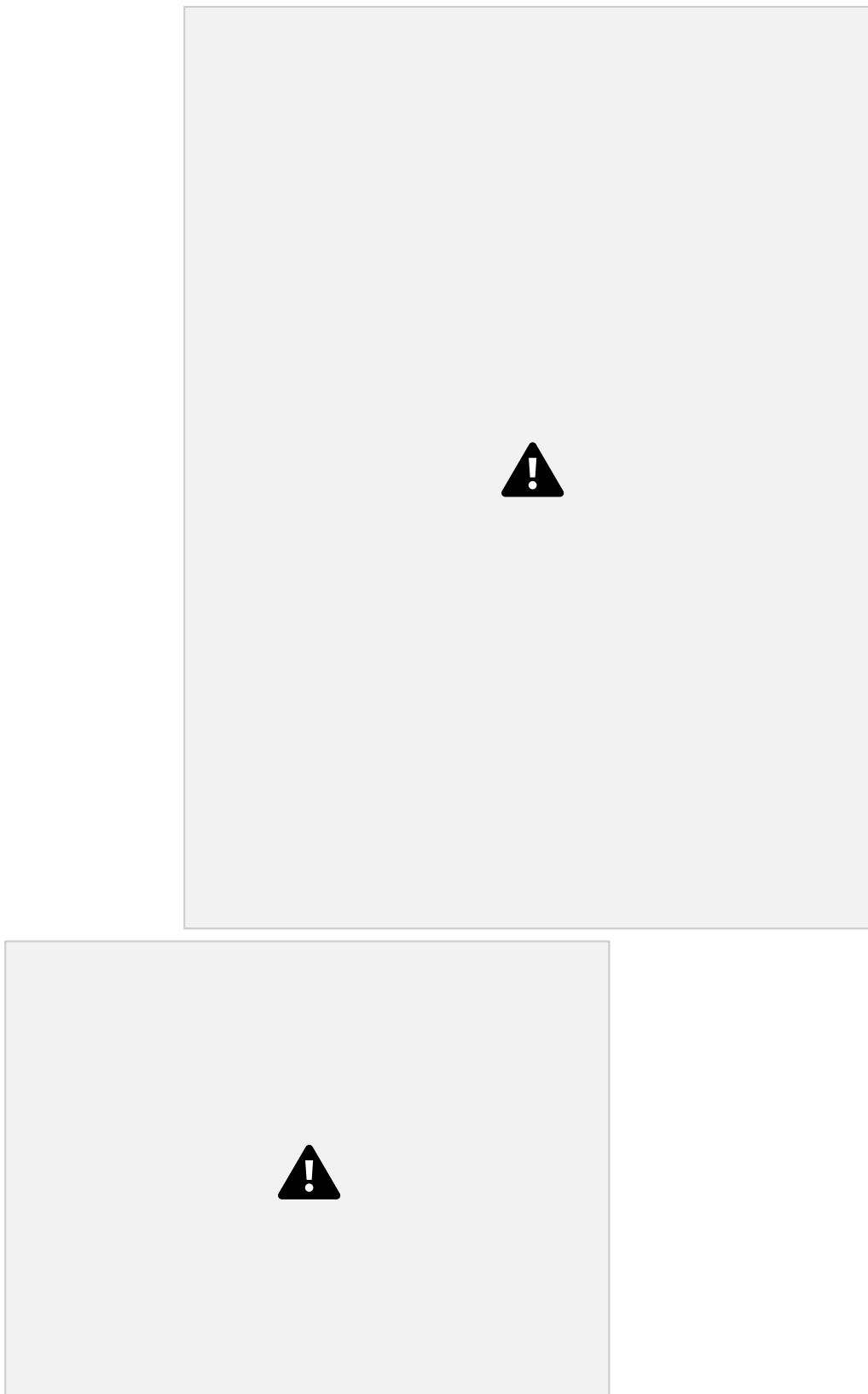
```

Example: Insertion of either '12' or '18' makes AVL Tree imbalance



(b) After rotation

the single rotation with left of 20 as shown below.





This can also be done by performing single rotation with left and then single rotation with right.



Routine to Perform Double Rotation with Right:

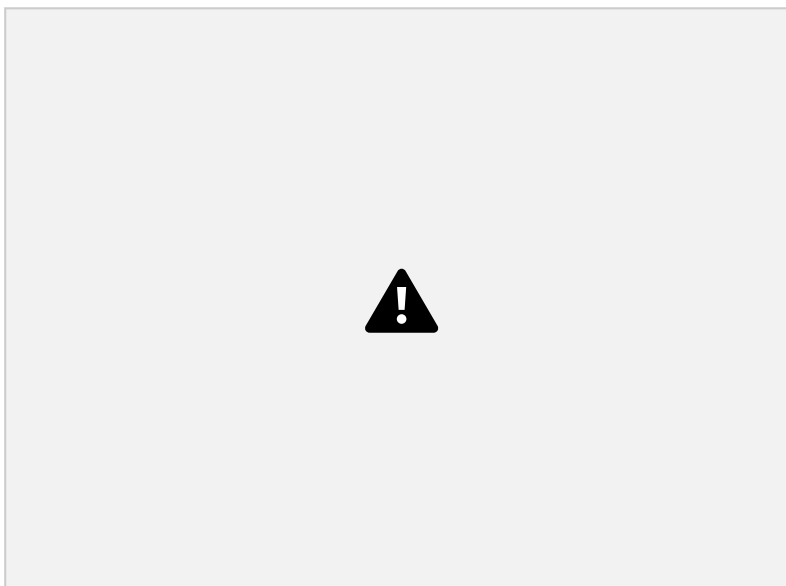
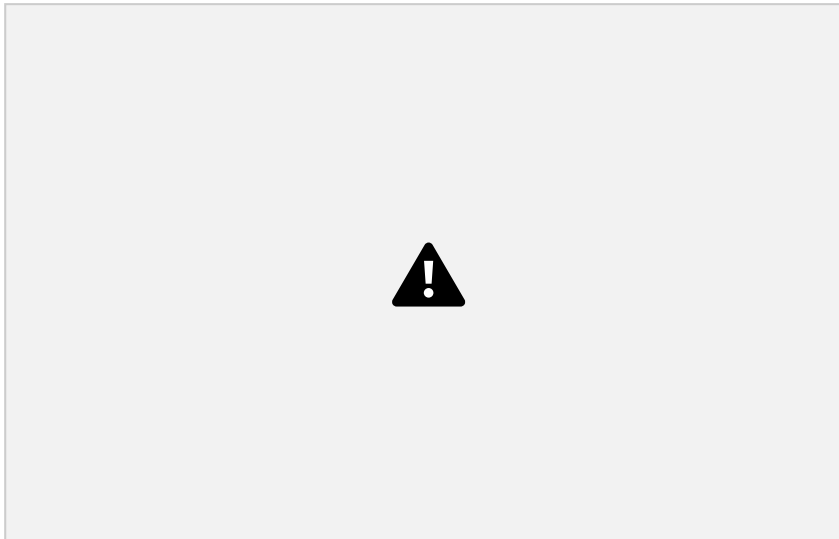
Double Rotate with Right (Position K1)

{

/* Rotation Between K2 & K3 */

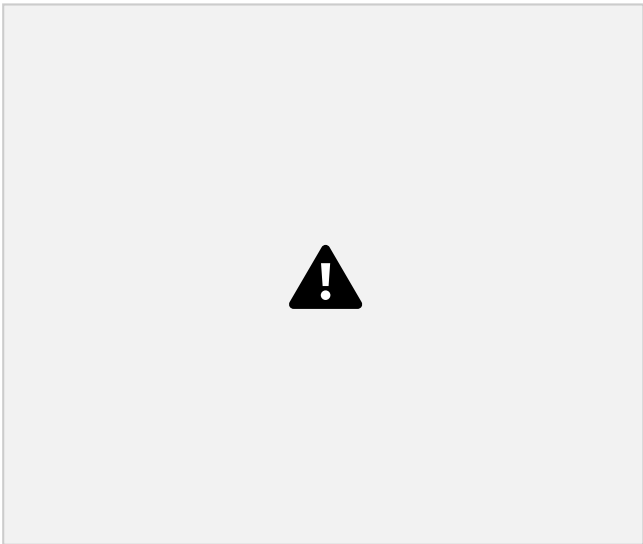
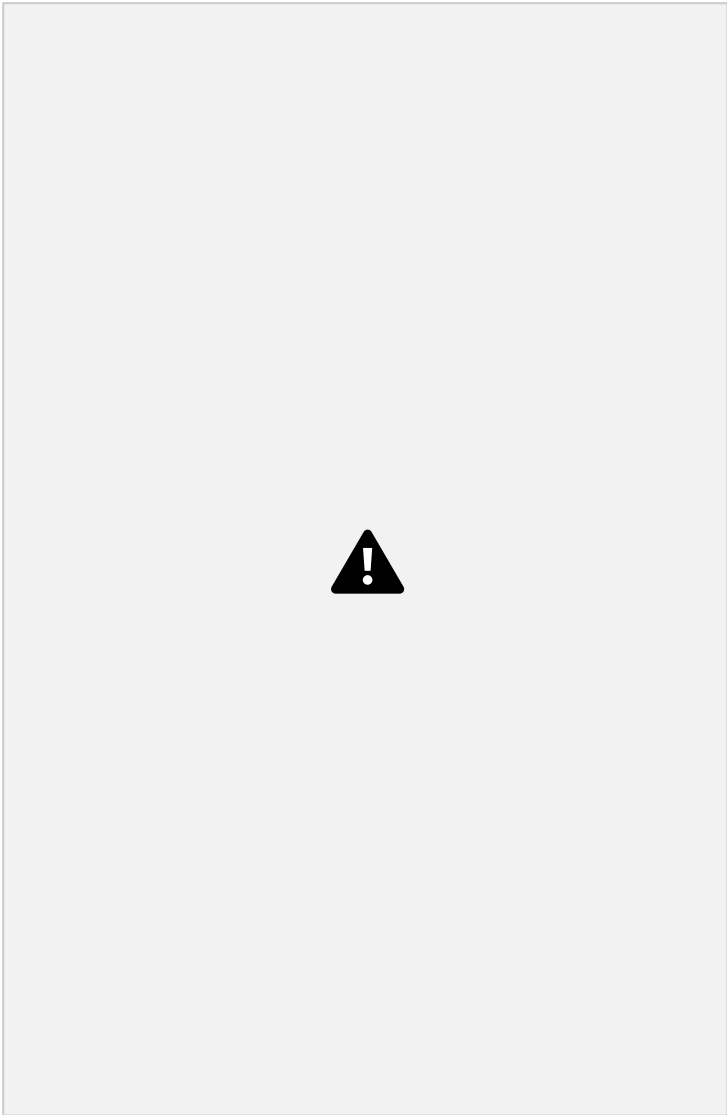
K1 -> Right = Single Rotate With Left (K1 -> Right);

```
/* Rotation Between K1 & K2 */  
    return Single RotateWithRight (K1);  
}
```



(b) After

This can be done by performing single rotation with left of '15' and then performing the single rotation with right of '10' as shown below



Routine to Insert in an AVL Tree : struct

```
Node *insert(struct Node* node, int key) {  
    if (node == NULL)  
        return createNode(key);
```

Meenakshi Page 37

```
    if (key < node->key)  
        node->left = insert(node->left, key);  
    else if (key > node->key)  
        node->right = insert(node->right, key);  
  
    node->height = 1 + max(getHeight(node->left), getHeight(node->right));  
    int bf = getBalanceFactor(node);
```

```
    // Left Left Case
```

```
    if(bf>1 && key < node->left->key){  
        return rightRotate(node);  
    }
```

```
    // Right Right Case
```

```
    if(bf<-1 && key > node->right->key){  
        return leftRotate(node);  
    }
```

```
    // Left Right Case
```

```
    if(bf>1 && key > node->left->key){  
        node->left = leftRotate(node->left);  
        return rightRotate(node);  
    }
```

```
    // Right Left Case
```

```
    if(bf<-1 && key < node->right->key){  
        node->right = rightRotate(node->right);  
        return leftRotate(node);  
    }
```

```
    return node;
```

```
}
```

3.7 Priority Queue (Heaps) – Binary Heap

- In an priority queue, an element with high priority is served before an element with low priority.
- If two elements have the same priority, they are served according to their order in the queue.

Two types of priority queue

1. Max Priority Queue
2. Min Priority Queue

Max Priority Queue

In Max Priority Queue, elements are inserted in the order in which they arrive they queue and always maximum value is removed first from the queue. E.x : insert in order 8, 3, 2, 5 removed in the order 8, 5, 3, 2

Min Priority Queue

Min Priority Queue is similar to Max priority queue except removing maximum elements first, we remove min. element first in min priority queue. **3.8 BINARY**

HEAP

- The efficient way of implementing priority queue is Binary Heap. •

Binary heap is merely referred as Heaps

Heap have two properties namely

1. Structure property
2. Heap order property.

Structure Property

- A heap should be complete binary tree, which is a completely filled binary tree with the possible exception of the bottom level, which is filled from left to right.
- A complete binary tree of height H has between 2^H and $2^{H+1} - 1$ nodes. • For example, if the height is 3. Then the number of nodes will be between 8 and

15. (ie) (2^3 and 2^4-1).

- For any element in array position i , the left child is in position $2i$, the right child is in position $2i + 1$, and the parent is in $i/2$.

Meenakshi Page 39

- As it is represented as array it doesn't require pointers and also the operations required to traverse the tree are extremely simple and fast.
- But the only disadvantage is to specify the maximum heap size in advance.

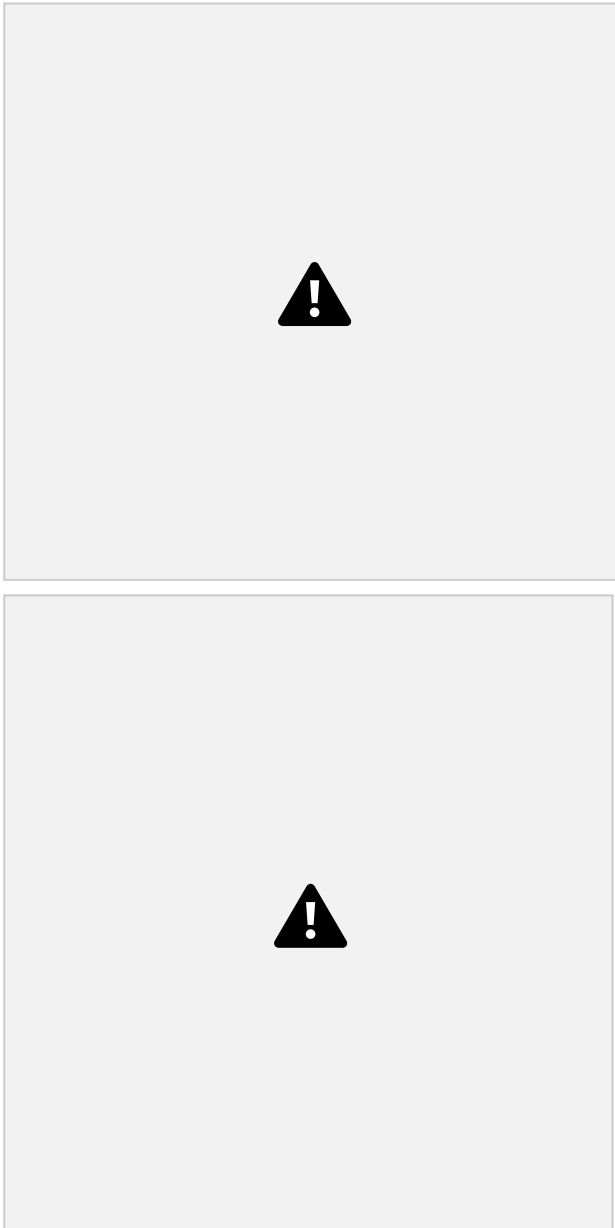


Heap Order Property

- In a heap, for every node X , the key in the parent of X is smaller than (or equal to) the key in X .

Meenakshi Page 40

- This property allows the delete operations to be performed quickly as the minimum element can always be found at the root.
- Thus, we get the FindMin operation in constant time.



Declaration for priority queue

```
struct Heapstruct
```

```
{
```

```
int capacity;
```

```
int size;
```

```
int *Elements;  
}; struct Heapstruct;
```

Meenakshi Page 41

```
typedef struct Heapstruct * PriorityQueue;
```

Initialization

PriorityQueue Initialize (int MaxElements)

```
{  
PriorityQueue H;  
H = malloc (sizeof (Struct Heapstruct));  
H -> Capacity = MaxElements;  
H -> size = 0;  
H -> Elements [0] = MinData;  
return H;  
}
```

Basic Heap Operations

To perform the insert and DeleteMin operations ensure that the heap order property is maintained.

Insert Operation

- To insert an element X into the heap, we create a hole in the next available location, otherwise the tree will not be complete.
- If X can be placed in the hole without violating heap order, then place the element X there itself.
- Otherwise, we slide the element that is in the hole's parent node into the hole, thus bubbling the hole up toward the root.
- This process continues until X can be placed in the hole.
- This general strategy is known as Percolate up, in which the new element is percolated up the heap until the correct location is found.

Routine to insert into a Binary Heap

```
void insert (int X, PriorityQueue H)
```

```
{
```

```
int i;
```

Meenakshi Page 42

```
if (Isfull (H))
```

```
{
```

```
Error ("priority queue is full");
```

```
return;
```

```
}
```

```
for (i = ++H -> size; H -> Elements [i/2] > X; i/=2) /* If the parent value is > X,  
then place the element of parent node into the hole */.
```

```
H -> Elements [i] = H -> Elements [i/2];
```

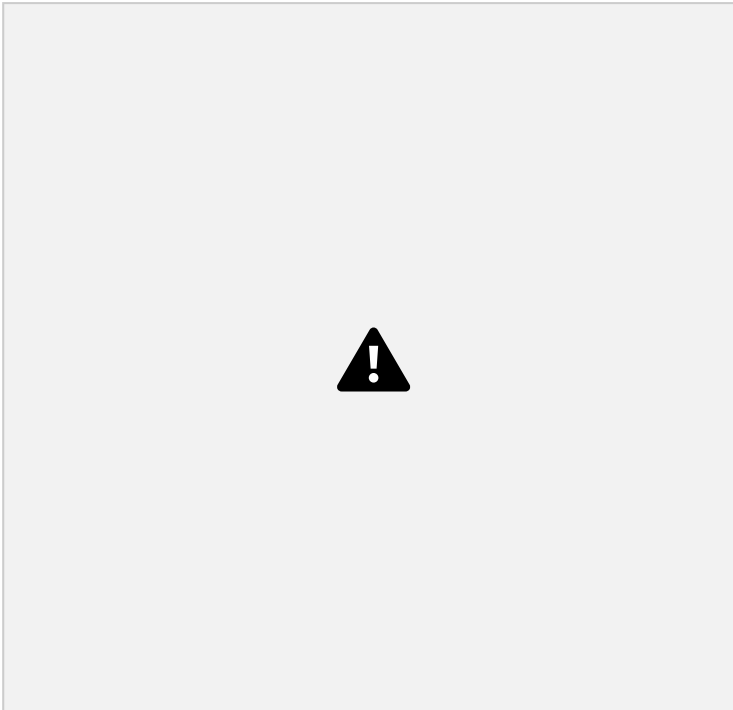
```
H -> Elements [i] = X; // otherwise, place it in the hole.
```

```
}
```



Meenakshi

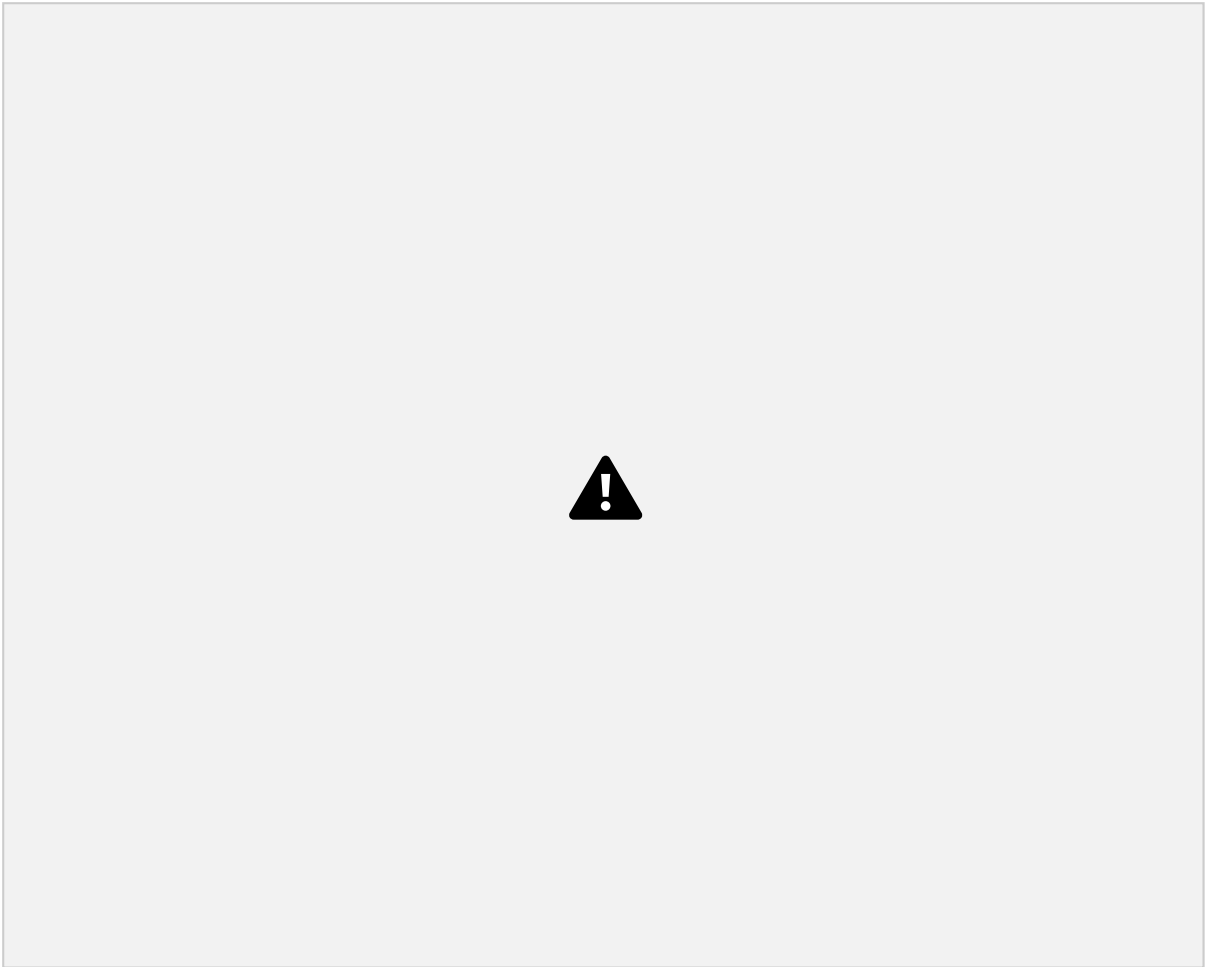




In Fig (d) the value 10 is placed in its correct location.

DeleteMin

- DeleteMin Operation is deleting the minimum element from the Heap. •
- In Binary heap the minimum element is found in the root.
- When this minimum is removed, a hole is created at the root. • Since the heap becomes one smaller, makes the last element X in the heap to move somewhere in the heap.
- If X can be placed in hole without violating heaporder property place it. • Otherwise, we slide the smaller of the hole's children into the hole, thus pushing the hole down one level.
- We repeat until X can be placed in the hole.
- This general strategy is known as percolate down.





Meenakshi Page

46





Meenakshi

Page 47

Routine to Perform Deletemin in a Binary Heap

```
int Deletemin (PriorityQueue H)
{
    int i, child;
    int MinElement, LastElement;
    if (IsEmpty (H))
    {
        Error ("Priority queue is Empty");
        return H -> Elements [0];
    }
}
```

```

MinElement = H -> Elements [1];
LastElement = H -> Elements [H -> size - 1];
for (i = 1; i * 2 <= H -> size; i = child)
{
/* Find Smaller Child */
child = i * 2;
if (child != H -> size && H -> Elements [child + 1] < H -> Elements [child])
child ++;
// Percolate one level down
if (LastElement > H -> Elements [child])
H -> Elements [i] = H -> Elements [child];
else
break ;
}
H -> Elements [i] = LastElement;
return MinElement;
}

```

APPLICATIONS OF HEAP

- To quickly find the smallest and largest element from a collection of items or array.

Meenakshi Page 48

- In the implementation of Priority queue in graph algorithms like Dijkstra's algorithm (shortest path), Prim's algorithm (minimum spanning tree) and Huffman encoding (data compression).
- In order to overcome the Worst Case Complexity of Quick Sort algorithm from $O(n^2)$ to $O(n \log(n))$ in Heap Sort.
- For finding the order in statistics.
- Systems concerned with security and embedded system such as Linux Kernel uses Heap Sort because of the $O(n \log(n))$.

